Le Shelter Foundation Website

Security, Architecture, and Resilience Assessment

Assessment date: 26 August 2026 Repository: Le-Shelter Assessment type: Read-only source review and configuration assessment Audience: Technical maintainers, NGO leadership, and delivery partners

Executive summary

The repository is a client-rendered static website composed of 11 HTML pages, shared CSS, two small JavaScript files, Tailwind CSS as a build-time dependency, approx. 99.5 MB of images, and one external Google Apps Script checkout integration. It is not a Next.js or Supabase application. There is no first-party server, database layer, authentication system, or SQL in the reviewed repository.

The informational pages have a relatively small attack surface. The shop changes the risk profile because it processes orders without a trusted server boundary. Browser-controlled cart data is rendered through innerHTML, and complete orders are posted directly to a public Google Apps Script endpoint. Product names, prices, quantities, totals, customer details, and dates can all be forged outside the user interface.

The two highest-priority corrections are:

1. Replace the public browser-to-spreadsheet integration with a controlled API that validates requests, recalculates prices, rate-limits abuse, neutralizes spreadsheet formulas, and returns a verifiable result.
2. Replace the localStorage to innerHTML cart-rendering path with DOM-safe rendering and strict cart-schema validation.

Risk summary

ID	Finding	Severity	Confidence
F-01	Public unauthenticated order endpoint and forged order values	High	High
F-02	Persistent DOM-XSS gadget in cart rendering	High	High
F-03	Automated abuse and denial-of-service exposure	High	Medium-high
F-04	Spreadsheet formula injection	High	Medium
F-05	Checkout reports opaque failures as success	Medium	High
F-06	No deployable security-header policy in the repository	Medium	Medium
F-07	Personal-data and child-image governance gaps	Medium	Medium
F-08	Third-party availability and privacy dependencies	Low-medium	High

1. Overview of the current security posture

1.1 Architecture and data flow

The browser downloads static HTML, local CSS, local images, Google Fonts, and Font Awesome. The shop stores a cart in localStorage. Checkout sends personally identifiable information and the entire client-calculated cart directly to an embedded Google Apps Script URL, which presumably writes to a Google Sheet.

Primary data flow:

Visitor browser to static site to localStorage cart to public Apps Script to presumed Google Sheet

The critical trust boundaries are:

- localStorage into the DOM: stored values are browser-controlled and are not validated.
- Browser into Apps Script: client-side validation is the only visible control.
- Apps Script into Google Sheets: handler code, access permissions, retention, and formula handling are outside the repository.
-

Website into third parties: fonts, icons, placeholder images, maps, social networks, and Google receive connection metadata.

The contact and newsletter forms do not have a functional submission target. The donation form calls an undefined handleDonationSubmit function. These are reliability defects and may mislead visitors about whether information was received.

1.2 Positive controls

- No passwords, database credentials, private API keys, Supabase keys, or other obvious secrets were found.
- No SQL or database client exists, so SQL injection is not present in the reviewed code.
- No authentication or session system exists, so authentication bypass is not applicable.
- Most external links opened in a new tab explicitly use noopener and often noreferrer.
- Font Awesome is pinned to version 6.6.0 and has Subresource Integrity.
- Most simple dynamic values are written with textContent.
- An offline npm audit reported zero known vulnerabilities across 73 dependencies. An online advisory scan should still run in CI.

1.3 Overall assessment

Area	Rating	Basis
Static informational content	Low-moderate	Small runtime surface; defense-in-depth headers are not codified
Shop and order integrity	High risk	Entire order and price calculation are client-controlled
XSS resistance	Moderate-high risk	Unsafe persisted data flow into HTML and event-handler contexts
DoS and automated abuse	High risk	Public write endpoint with no visible rate or size controls
Privacy and safeguarding	Moderate risk	PII and child imagery without visible governance documentation
Dependencies	Low risk	One build dependency; offline audit clean
Maintainability	High risk	Repeated layouts and scripts across 11 HTML files
Performance	Moderate-high risk	Approximately 99.5 MB of image assets

2. Specific vulnerabilities

F-01 - Public unauthenticated order submission and forged order data

Severity: High Evidence: shop.html:686-726

The Google Apps Script URL is embedded in public JavaScript. The browser constructs and submits the complete order, including product name, price, quantity, total, customer name, destination, phone, and date. Anyone can copy the endpoint and submit arbitrary payloads without using the website.

Root cause: The browser is treated as an authoritative source rather than an untrusted client.

Impact: Fake orders, fraudulent prices, polluted reporting, staff workload, quota exhaustion, and potential abuse of permissions delegated to the Apps Script owner.

F-02 - Persistent DOM XSS through cart storage

Severity: High Evidence: shop.html:492, shop.html:549-571

Cart objects are parsed directly from localStorage. Their names, identifiers, colors, sizes, prices, and quantities are interpolated into an HTML string. The identifier is also inserted into inline onclick JavaScript. The combined string is assigned to innerHTML.

There are two distinct injection contexts: HTML body content and JavaScript string content inside an event attribute. A single generic escaping function cannot safely cover both contexts.

Direct exploitation currently requires a way to modify the victim's same-origin storage, which limits immediate reach. Nevertheless, the code is a persistent XSS gadget. A future query-string cart import, product API, CMS integration, compromised dependency, shared device, or browser extension could activate it remotely.

Impact: Page defacement, replacement of donation instructions, fake checkout UI, theft of future same-origin data, and persistence across reloads.

F-03 - Denial-of-service and automated abuse

Severity: High Evidence: Direct public POST at shop.html:721-726; no server controls present in repository.

No CAPTCHA, server-side rate limit, request-size limit, cart-line limit, idempotency control, origin validation, or queue is visible. Attackers can automate submissions directly against Apps Script. Google quota exhaustion would create an uncontrolled outage and may cause genuine orders to be lost.

The static host is naturally more resilient than a dynamic application, but CDN and hosting controls were not available for inspection.

F-04 - Spreadsheet formula injection

Severity: High Confidence: Medium because the Apps Script source is unavailable.

Customer and cart fields are sent to a presumed spreadsheet. If Apps Script writes these as interpreted cell values, strings beginning with =, +, -, @, tab, or carriage return can become formulas. This can lead to spreadsheet data exfiltration or malicious behavior when staff review or export orders.

The server must force all untrusted cells to literal text. Browser validation is not sufficient.

F-05 - Checkout reports failure as success

Severity: Medium Evidence: shop.html:721-734

Checkout uses Fetch with mode: "no-cors" and treats every resolved promise as success. A no-CORS response is opaque: JavaScript cannot inspect its HTTP status, headers, or body. A server-side rejection or error can therefore be shown as a successful order, after which the cart is deleted.

Impact: Lost orders, false confirmation, customer confusion, and no reliable reconciliation or retry mechanism.

F-06 - Security headers are not codified

Severity: Medium

No hosting configuration in the repository defines Content Security Policy, HSTS, MIME sniffing protection, referrer policy, permissions policy, or anti-framing behavior. The live host may add them externally, so production headers must be independently tested.

Current inline scripts, inline onclick and onsubmit attributes, inline style blocks, and style attributes make a strict CSP difficult to deploy. The unsafe inline handlers should be removed before enforcement.

F-07 - Personal-data and beneficiary-image governance

Severity: Medium

Checkout collects names, telephone numbers, and delivery destinations, but no privacy notice, purpose statement, retention period, deletion process, processor disclosure, or data-subject contact appears near the form. A spreadsheet can also be widely shared or retained indefinitely.

The gallery contains many photographs of NGO beneficiaries, including children. The repository cannot demonstrate publication consent, retention decisions, safeguarding review, or removal of EXIF location/device metadata.

This review does not determine violation of a particular law. Obligations depend on the NGO's operations and user locations, but the governance gap is material.

F-08 - Third-party dependencies

Severity: Low-medium

The pages connect to Google Fonts, cdnjs, Google services, social platforms, and an external placeholder-image service. This discloses connection metadata, broadens CSP, and creates availability dependencies. Font Awesome has SRI; dynamically generated Google font CSS does not.

Explicitly not found

- No SQL injection: no SQL or database code exists.
- No authentication bypass: no authentication system exists.
- No session fixation or cookie weakness: no sessions or cookies are used.
- No IDOR/BOLA: there are no authenticated resources or object endpoints.
- Traditional CSRF is not the central issue: the order endpoint accepts unauthenticated direct requests, making direct abuse easier than cross-site request forgery.

3. Recommended fixes

3.1 Replace HTML-string rendering

Build DOM nodes, assign untrusted text using textContent, and attach listeners with addEventListener.

```
function renderCartItem(item) {
  const row = document.createElement('div');
  row.className = 'cart-row';

  const name = document.createElement('p');
  name.textContent = item.name;

  const variant = document.createElement('p');
  variant.textContent = ${item.color ?? ''} ${item.size ?? ''};

  const remove = document.createElement('button');
  remove.type = 'button';
  remove.textContent = 'Remove';
  remove.addEventListener('click', () => removeFromCart(item.id));

  row.append(name, variant, remove);
  return row;
}

cartItems.replaceChildren(...cart.map(renderCartItem));
```

Store only product IDs, selected variants, and bounded quantities. Validate the parsed storage value before using it. On any parse or schema failure, remove it and use an empty cart.

3.2 Add a controlled order API

Adopt this data flow:

Browser to POST /api/orders to validate, rate-limit, reprice to queue or Google service

The API must accept only product IDs, variants, quantities, and necessary customer data. It must retrieve product names and prices from a server-owned catalog and calculate all totals itself.

```
const catalog = {
  tshirt: { name: 'Organic T-Shirt', price: 45 },
  tote: { name: 'Canvas Tote Bag', price: 35 },
  cap: { name: 'Branded Cap', price: 30 }
};

function normalizeItem(input) {
  const product = catalog[input.productId];
  const quantity = Number(input.quantity);

  if (!product || !Number.isInteger(quantity) ||
      quantity < 1 || quantity > 20) {
    throw new Error('Invalid item');
  }

  return {
    productId: input.productId,
    name: product.name,
    unitPrice: product.price,
    quantity,
    total: product.price * quantity
  };
}
```

The browser should use a same-origin endpoint, a real JSON response, and an idempotency key. Remove mode: "no-cors".

```
const response = await fetch('/api/orders', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Idempotency-Key': crypto.randomUUID()
  },
  body: JSON.stringify({ name, phone, destination, items })
});

if (!response.ok) throw new Error(Order rejected: ${response.status});
const { orderId } = await response.json();
```

3.3 Rate, size, and bot controls

- Allow about five order attempts per IP per ten minutes with a small burst allowance.
- Add a secondary limit by normalized phone number.
- Reject request bodies larger than 16-32 KB before parsing.
- Limit textual fields, cart lines, and quantity per item.
- Require a server-verified Turnstile, hCaptcha, or equivalent token.
- Generate an order ID and enforce idempotency server-side.
- Apply short upstream timeouts and queue accepted orders before spreadsheet writes.
- Alert on abnormal submission and rejection volumes.
- Never use Google quota exhaustion as the primary abuse control.

3.4 Neutralize spreadsheet formulas

Write untrusted values as literal text and neutralize dangerous leading characters on the server.

```
function spreadsheetSafe(value, maxLength = 200) {
  let text = String(value ?? '')
    .replace(/[\u0000-\u0008\u000B\u000C\u000E-\u001F]/g, '')
    .slice(0, maxLength);

  if (/^[\s]*[=+\-@]/.test(text)) text = '${text};
  return text;
}
```

Keep the sheet private, restrict named staff access, protect administrative columns, and re-sanitize data during CSV export.

3.5 Deploy CSP and security headers

First extract inline JavaScript into local files and remove inline event handlers. Begin with Content-Security-Policy-Report-Only, review violations, and then enforce:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'
  https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src
  'self' data:; connect-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors
  'none'; form-action 'self'; upgrade-insecure-requests
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()
Cross-Origin-Opener-Policy: same-origin
```

Only add HSTS preload after confirming all subdomains permanently support HTTPS. Do not solve CSP violations with wildcards, unsafe-eval, or permanent unsafe-inline.

3.6 Privacy and safeguarding

- Put a privacy notice beside checkout describing purpose, storage, processors, retention, deletion, and contact procedures.
- Minimize data fields and delete stale or unsuccessful orders on a documented schedule.
- Restrict the spreadsheet to named accounts protected with MFA.
- Review the Apps Script owner's scopes, recovery controls, and access logs.
- Strip EXIF metadata before publishing photographs.
- Maintain documented guardian/subject consent for identifiable children.
- Avoid detailed school/location information that creates safeguarding risk.
- Establish an incident procedure for compromised donation instructions or social accounts.

4. Website security, engineering, and scalability practices

4.1 Structural problems and duplicated code

Navigation, headers, donation modal, contact information, social links, newsletter form, footer, menu logic, and modal logic are copied across most pages. A security fix can therefore be applied to one page while stale vulnerable copies remain elsewhere.

The shop declares the same top-level constants in multiple classic script blocks around shop.html:795. Redeclaring top-level const values can cause syntax errors. The expired countdown around shop.html:858 also writes to a potentially absent element. The donation page references an undefined submit handler, and at least one image path appears inconsistent with the repository layout.

Use a static-site generator or small template build with shared header, footer, modal, and script modules. A full dynamic framework is unnecessary unless authenticated administration or trusted server workflows are introduced.

4.2 Performance bottlenecks

The image directory contains approx. 99.5 MB across 146 files. Several images are over 1 MB and one is approx. 4.2 MB. This increases mobile data usage, decoding CPU, memory consumption, page latency, hosting bandwidth, and vulnerability to traffic-driven bandwidth exhaustion.

Create responsive AVIF/WebP renditions, keep JPEG fallbacks, add explicit dimensions, lazy-load below-the-fold media, set decoding="async", remove unused duplicates, and cache hashed assets through a CDN.

4.3 Third-party reduction

Self-host stable fonts, icons, and production product images. If a CDN remains, pin immutable versions, use SRI where possible, prohibit unversioned executable JavaScript, and monitor failed loads.

4.4 Secure build and deployment

CI should run dependency review, an online npm advisory scan, secret scanning, HTML validation, linting, broken-link checks, CSP/header tests, and image-size budgets. Deploy only generated static output; do not publish .git, node_modules, local settings, backup files, or source maps unintentionally.

4.5 Authorized production-style testing

On an isolated staging deployment, test DOM injection sinks, formula-leading values, oversized payloads, unknown products, modified prices, negative/fractional/huge quantities, replayed orders, burst rate limits, Google quota failures, CSP violations, TLS, headers, directory listings, and exposure of repository files.

Suitable tools include OWASP ZAP, Burp Suite, Mozilla Observatory, SecurityHeaders.com, Lighthouse, a TLS scanner, dependency review, and secret scanning. Do not load-test the live Apps Script without explicit authorization, quotas, monitoring, and rollback controls.

Remediation roadmap

Immediate - 1 to 3 days

1. Review, rotate, or disable the exposed Apps Script deployment until its handler and permissions are verified.
2. Stop accepting client-provided prices, product names, dates, and totals.
3. Replace the innerHTML cart renderer and inline event handlers.
4. Add server-side body, field, item-count, and quantity limits.
5. Neutralize formula-capable spreadsheet input.
6. Restrict Sheet and Apps Script administration and enforce MFA.

Short term - 1 to 2 weeks

1. Put checkout behind a controlled same-origin API.
2. Add bot verification, rate limiting, idempotency, and verifiable responses.
3. Extract inline scripts and introduce CSP Report-Only.
4. Add the remaining security headers.
5. Add a privacy/retention notice and beneficiary-image safeguards.
6. Consolidate duplicated layout and scripts.

Medium term - 2 to 6 weeks

1. Optimize and deduplicate images.
2. Self-host fonts, icons, and product images.
3. Add CI security gates and automated dependency updates.
4. Add order-abuse monitoring and incident response.
5. Run an authorized staging penetration test.
6. Enforce CSP after report-only validation.

Scope, limitations, and verification

The review covered tracked HTML, JavaScript, CSS/build configuration, package metadata, static assets, and the browser-side order flow. It was read-only. No requests were sent to the exposed order endpoint, no production load testing occurred, and no user or third-party data was accessed.

Not provided for assessment: hosting/CDN configuration, live response headers, DNS/TLS configuration, Google Apps Script source, Apps Script deployment permissions, Google Sheet sharing settings, operational logs, privacy policies, image consent records, or incident-response procedures. Findings involving those components must be verified by reviewing them directly.

An offline npm audit found no known vulnerable dependencies. Because advisory databases change, run an online audit in CI before each deployment.

The report distinguishes confirmed source findings from conditional risks, includes all requested security categories, and provides tailored remediation rather than framework-generic advice.

References

1. OWASP Content Security Policy Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
2. OWASP HTTP Headers Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html
3. OWASP CSV/Formula Injection: https://owasp.org/www-community/attacks/CSV_Injection
4. MDN Response type and opaque responses: https://developer.mozilla.org/en-US/docs/Web/API/Response/type
5. Google Apps Script Web Apps: https://developers.google.com/apps-script/guides/web
6. OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/

Conclusion: The static informational site has a limited attack surface, but the shop currently behaves as an unauthenticated public order-processing application without a trusted server boundary. A controlled API, authoritative server pricing, safe DOM construction, abuse controls, spreadsheet hardening, CSP, and consolidated templates will materially improve security, reliability, scalability, and donor trust.